

Curso Profesional de JasperReports 6.20.0 Community

Módulo 5 — Diseño avanzado

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Módulo 5 — Diseño avanzado

Proyecto acumulativo: **EditorialReports**. Baseline: M4/4.6 validado E2E.

Criterio editorial: los objetivos y la organización proceden del material original del M5. Las afirmaciones técnicas se han contrastado con el código ejecutable y JasperReports 6.20.0; cuando la fuente era incorrecta se conserva el objetivo pedagógico y se corrige la implementación.

Punto 5.1 — Subreportes

Objetivos de aprendizaje

- Comprender el concepto de subreporte como informe anidado dentro de otro.
- Declarar un subreporte en el JRXML con el elemento `subreport`.
- Pasar parámetros y conexiones al subreporte mediante `subreportParameter` y `connectionExpression`.
- Construir una relación maestro-detalle entre dos informes.
- Depurar errores de resolución de subreportes.
- Documentar los subreportes del proyecto EditorialReports.

Parte teórica

Bloque 1 — Concepto de subreporte

Un subreporte es un informe que se ejecuta dentro de otro informe. El informe principal se denomina maestro y el informe anidado se denomina subreporte o detalle. El subreporte se declara dentro de una banda del informe maestro mediante el elemento `subreport` y se ejecuta en el momento de la emisión de esa banda. El resultado del subreporte se incrusta en el documento del informe maestro como si fuera parte de él. Esta técnica permite construir informes complejos con estructuras jerárquicas que no pueden representarse en una sola consulta SQL. La relación entre el maestro y el subreporte se denomina relación maestro-detalle.

XML

```
<subreport>
  <reportElement x="0" y="0" width="555" height="20" uuid="..." />
  <subreportExpression><![CDATA["reports/subreporte.jasper"]]></subreportExpression>
</subreport>
```

Línea 1	<code><subreport></code> → declara un elemento de subreporte. Puede colocarse en cualquier banda del informe maestro.
Línea 2	<code><reportElement x="0" y="0" width="555" height="20" uuid="..." /></code> → define la posición y el tamaño del elemento dentro de la banda.
Línea 3	<code><subreportExpression><![CDATA["reports/subreporte.jasper"]]></subreportExpression></code> → expresión que devuelve la ruta del artefacto compilado del subreporte. El motor carga el subreporte y lo ejecuta en el momento de la emisión.

Los subreportes se utilizan cuando el informe debe mostrar datos relacionados que no pueden obtenerse con una única consulta. Un informe de facturas con sus líneas de detalle es un ejemplo clásico: el maestro muestra las facturas y el subreporte muestra las líneas de cada factura. Un informe de libros con sus ventas individuales es otro ejemplo: el maestro muestra los libros y el subreporte muestra las ventas de cada libro. La separación entre maestro y subreporte permite que cada nivel tenga su propia consulta, su propia fuente de datos y sus propias bandas. La composición de los dos niveles produce un documento con estructura jerárquica.

TEXT

ESTRUCTURA MAESTRO-DETALLE

Informe maestro:

Título del libro: Cien años de sol.
Unidades vendidas: 8
Importe total: 159,60 €

Subreporte: ventas individuales		
Fecha	Cant.	Precio
2026-09-01	3	19,95 €
2026-09-05	5	19,95 €

Título del libro: Rayuela
Unidades vendidas: 6
Importe total: 135,00 €

Subreporte: ventas individuales		
Fecha	Cant.	Precio
2026-09-03	2	22,50 €
2026-09-07	4	22,50 €

Qué representa el diagrama: la estructura maestro-detalle. El maestro emite una sección por cada libro y el subreporte emite una fila por cada venta del libro.

Por qué es relevante: permite construir informes jerárquicos sin necesidad de escribir consultas complejas con JOIN y GROUP BY.

Bloque 2 — Declaración del subreporte en el JRXML

Un subreporte se declara en el informe maestro con el elemento `subreport`. Este elemento contiene un bloque `reportElement` con la posición y el tamaño, un bloque `subreportExpression` con la ruta del artefacto compilado y, opcionalmente, un bloque `connectionExpression` con la conexión que se pasa al subreporte. El subreporte se ejecuta cada vez que la banda que lo contiene se emite. Si el subreporte no encuentra datos, no emite ninguna banda pero no produce error. Si el subreporte no se puede cargar, el motor lanza una excepción en el momento de la emisión.

XML

```
<subreport>
  <reportElement x="0" y="20" width="555" height="30" uuid="..." />
  <connectionExpression><![CDATA[{$REPORT_CONNECTION}]]></connectionExpression>
  <subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression>
</subreport>
```

Línea 1	<code><subreport></code> → declara el elemento de subreporte.
Línea 2	<code><reportElement x="0" y="20" width="555" height="30" uuid="..." /></code> → posición y tamaño del elemento dentro de la banda.
Línea 3	<code><connectionExpression><![CDATA[{\$REPORT_CONNECTION}]]></connectionExpression></code> → expresión que devuelve la conexión que el subreporte utilizará. El parámetro interno <code>REPORT_CONNECTION</code> contiene la conexión que el maestro recibió. El subreporte la hereda y ejecuta su propia consulta.
Línea 4	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → expresión que devuelve la ruta del artefacto compilado del subreporte.

La conexión no es la única forma de alimentar un subreporte. El subreporte puede recibir una fuente de datos propia mediante el elemento `dataSourceExpression` o heredar la fuente de datos del maestro. La elección entre las tres formas depende de si el subreporte necesita una consulta distinta, la misma conexión o los mismos datos. La conexión es la forma más habitual cuando el subreporte ejecuta su propia consulta SQL. La fuente de datos propia se utiliza cuando el subreporte recibe una colección de objetos Java. La herencia de la fuente de datos se utiliza cuando el subreporte itera sobre los mismos registros que el maestro.

TEXT

FORMAS DE ALIMENTAR UN SUBREPORT

```
connectionExpression:
- El subreporte ejecuta su propia consulta SQL.
- Hereda la conexión del maestro.
- Adecuado para subreportes con consultas filtradas.

dataSourceExpression:
- El subreporte recibe una fuente de datos propia.
- Puede ser un JRBeanCollectionDataSource, un JRMapCollectionDataSource, etc.
- Adecuado para subreportes que reciben colecciones de objetos.

Sin conexión ni fuente de datos:
- El subreporte hereda la fuente de datos del maestro.
- El subreporte itera sobre los mismos registros que el maestro.
- Adecuado para subreportes que comparten los datos del maestro.
```

Qué representa el diagrama: las tres formas de alimentar un subreporte. La elección depende del origen de los datos del subreporte.

Por qué es relevante: permite elegir la forma adecuada según la naturaleza del subreporte y su origen de datos.

Bloque 3 — Paso de parámetros al subreporte

Los parámetros que el subreporte necesita deben pasarse explícitamente desde el maestro. El paso se realiza con el elemento `subreportParameter` que se declara dentro del elemento `subreport`. Cada `subreportParameter` contiene un atributo `name` con el nombre del parámetro en el subreporte y una expresión `subreportParameterExpression` con el valor que el maestro proporciona. El nombre del parámetro debe coincidir exactamente con el declarado en el subreporte. La expresión puede contener campos del maestro, parámetros del maestro, variables del maestro o literales.

XML

```
<subreportParameter name="tituloLibro">
  <subreportParameterExpression><![CDATA[{$F{titulo}}]]></subreportParameterExpression>
</subreportParameter>
```

Línea 1	<code><subreportParameter name="tituloLibro"></code> → declara el paso de un parámetro llamado <code>tituloLibro</code> al subreporte. El nombre debe coincidir con el declarado en el subreporte.
Línea 2	<code><subreportParameterExpression><![CDATA[<code>\$F{titulo}</code>]]></subreportParameterExpression></code> → expresión que devuelve el valor que el maestro proporciona. En este caso, el campo <code>titulo</code> del registro actual del maestro.

El paso de parámetros es la forma habitual de comunicar el maestro y el subreporte. El maestro proporciona los valores que el subreporte utiliza en su consulta o en sus expresiones. Un subreporte que muestra las ventas de un libro recibe el título del libro como parámetro y lo utiliza en la cláusula `WHERE` de su consulta. La sustitución del parámetro en la consulta del subreporte se realiza con la misma sintaxis `$P{}` que en el informe principal. La coherencia entre el nombre del parámetro en el maestro y en el subreporte es condición necesaria para que el paso funcione.

TEXT

PASO DE PARÁMETROS DEL MAESTRO AL SUBREPORT

Maestro:

```
<subreportParameter name="tituloLibro">
  <subreportParameterExpression>$F{titulo}</subreportParameterExpression>
</subreportParameter>
```

Subreporte:

```
<parameter name="tituloLibro" class="java.lang.String"/>
<queryString>
  SELECT * FROM ventas WHERE titulo_libro = $P{tituloLibro}
</queryString>
```

El maestro pasa el valor del campo `titulo` del registro actual.
El subreporte lo recibe y lo utiliza en su consulta.

Qué representa el diagrama: el paso de un parámetro del maestro al subreporte y su uso en la consulta del subreporte. La comunicación se realiza por nombre.

Por qué es relevante: permite construir subreportes que dependen de los valores del registro actual del maestro.

Bloque 4 — Subreporte con conexión JDBC

El caso más habitual de subreporte es el que se alimenta con una conexión JDBC heredada del maestro. El maestro declara el elemento `connectionExpression` con el parámetro interno `REPORT_CONNECTION` y el subreporte ejecuta su propia consulta contra la misma conexión. Esta aproximación simplifica la gestión de la conexión porque el maestro la abre una sola vez y el subreporte la reutiliza. La contrapartida es que el subreporte comparte la conexión con el maestro y las consultas se ejecutan de forma secuencial.

XML

```
<subreport>
  <reportElement x="0" y="20" width="555" height="30" uuid="..." />
  <connectionExpression><![CDATA[$P{REPORT_CONNECTION}]]></connectionExpression>
  <subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression>
  <subreportParameter name="tituloLibro">
    <subreportParameterExpression><![CDATA[$F{titulo}]]></subreportParameterExpression>
  </subreportParameter>
</subreport>
```

Línea 1	<code><subreport></code> → declara el subreporte.
Línea 2	<code><reportElement ... /></code> → posición y tamaño del elemento.
Línea 3	<code><connectionExpression><![CDATA[<code>\$P{REPORT_CONNECTION}</code>]]></connectionExpression></code> → pasa la conexión del maestro al subreporte. El subreporte ejecutará su propia consulta contra esta conexión.
Línea 4	<code><subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression></code> → ruta del artefacto compilado del subreporte.
Línea 5-7	<code><subreportParameter name="tituloLibro"></code> → paso del parámetro <code>tituloLibro</code> al subreporte con el valor del campo <code>titulo</code> del maestro.

El subreporte que se alimenta con una conexión JDBC declara su propia consulta SQL con la sintaxis `$P{}`. La consulta puede incluir parámetros que el maestro ha proporcionado. El motor ejecuta la consulta del subreporte una vez por cada emisión de la banda que lo contiene. Si el maestro tiene 10 registros y el subreporte se declara en la banda de detalle, la consulta del subreporte se ejecuta 10 veces. Esta característica es la que permite construir relaciones maestro-detalle sin necesidad de escribir una consulta SQL con `JOIN` y `GROUP BY`. La contrapartida es el rendimiento: muchas consultas pequeñas pueden ser más lentas que una consulta grande.

TEXT

EJECUCIÓN DEL SUBREPORTE CON CONEXIÓN

Maestro emite el registro 1:

- El subreporte ejecuta su consulta con el parámetro del registro 1.
- El subreporte emite sus bandas para el registro 1.
- El resultado se incrusta en el documento del maestro.

Maestro emite el registro 2:

- El subreporte ejecuta su consulta con el parámetro del registro 2.
- El subreporte emite sus bandas para el registro 2.
- El resultado se incrusta en el documento del maestro.

Y así sucesivamente para cada registro del maestro.

Qué representa el diagrama: la ejecución del subreporte una vez por cada registro del maestro. Cada ejecución recibe el parámetro del registro actual.

Por qué es relevante: permite comprender el coste de rendimiento y el comportamiento del subreporte en la relación maestro-detalle.

Bloque 5 — Subreporte con fuente de datos propia

Un subreporte también puede alimentarse con una fuente de datos propia que el maestro proporciona. La fuente de datos se declara en el elemento `dataSourceExpression` y puede ser cualquier implementación de `JRDataSource`. El caso más habitual es el uso de `JRBeanCollectionDataSource` con una colección de objetos Java que el maestro construye a partir del registro actual. Esta aproximación es útil cuando el subreporte recibe datos que no residen en la base de datos, como una lista de elementos calculados.

XML

```
<subreport>
  <reportElement x="0" y="20" width="555" height="30" uuid="..." />
  <dataSourceExpression><![CDATA[new net.sf.jasperreports.engine.data.JRBeanCollectionDataSource($P{listaVentas})]]></
dataSourceExpression>
  <subreportExpression><![CDATA["reports/subinforme_ventas_detalle.jasper"]]></subreportExpression>
</subreport>
```

Línea 3

`<dataSourceExpression><![CDATA[new net.sf.jasperreports.engine.data.JRBeanCollectionDataSource($P{listaVentas})]]></dataSourceExpression>` → construye una fuente de datos a partir de la colección `listaVentas` que el maestro recibe como parámetro.

La fuente de datos propia permite que el subreporte reciba datos que el maestro ha procesado previamente. Un maestro puede consultar la base de datos, construir una lista de objetos y pasar la lista al subreporte. El subreporte itera sobre la lista como si fuera una consulta SQL. Esta aproximación es útil cuando la lógica de negocio requiere transformaciones que no pueden expresarse en SQL. La contrapartida es la complejidad del programa Java que debe construir la lista y pasarla al maestro. La elección entre conexión y fuente de datos propia depende del control que se necesite sobre los datos del subreporte.

TEXT

COMPARACIÓN ENTRE CONEXIÓN Y FUENTE DE DATOS PROPIA

Conexión JDBC:

- El subreporte ejecuta su propia consulta.
- El maestro solo pasa la conexión.
- El rendimiento depende del número de consultas.
- Adecuado cuando los datos residen en la base de datos.

Fuente de datos propia:

- El subreporte itera sobre una colección de objetos.
- El maestro construye la colección.
- El rendimiento depende del tamaño de la colección.
- Adecuado cuando los datos se construyen en memoria.

Qué representa el diagrama: las diferencias entre las dos formas de alimentar un subreporte. La elección depende del origen de los datos y del control que se necesite.

Por qué es relevante: permite elegir la forma adecuada según la naturaleza del subreporte y su rendimiento.

Resumen rápido de la teoría

- Un subreporte es un informe que se ejecuta dentro de otro informe.
- Se declara con el elemento `subreport` dentro de una banda del maestro.
- El elemento `subreportExpression` contiene la ruta del artefacto compilado.

- El elemento `connectionExpression` pasa la conexión del maestro al subreporte.
- El elemento `dataSourceExpression` pasa una fuente de datos propia al subreporte.
- Los parámetros se pasan con `subreportParameter` y `subreportParameterExpression`.
- El subreporte se ejecuta una vez por cada emisión de la banda que lo contiene.
- La coherencia de nombres entre el maestro y el subreporte es condición necesaria.

Punto 5.2 — Tablas

Objetivos de aprendizaje

- Comprender el elemento `table` y su diferencia con la banda `detail`.
- Declarar un dataset propio para la tabla con su consulta y sus campos.
- Configurar las columnas de la tabla con encabezado y celda de detalle.
- Asociar el dataset a la tabla mediante `datasetRun` y `connectionExpression`.
- Aplicar estilos a la tabla y a sus celdas.
- Documentar las tablas del proyecto `EditorialReports`.

Parte teórica

Bloque 1 — El elemento `table`

Una tabla en JasperReports es un componente que organiza datos en filas y columnas con un diseño propio, independiente de las bandas del informe. A diferencia de la banda `detail`, que emite una fila por cada registro del informe principal, la tabla tiene su propia fuente de datos, su propia estructura de columnas y su propio conjunto de bandas internas. La tabla se declara dentro de una banda del informe mediante el elemento `componentElement` que contiene un elemento `table`. La tabla puede tener una altura variable que se ajusta automáticamente al número de filas. El motor gestiona los saltos de página cuando la tabla no cabe completa.

XML

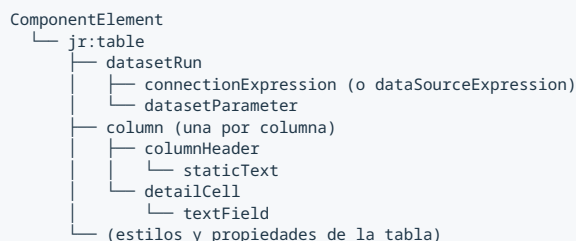
```
<componentElement>
  <reportElement x="0" y="100" width="555" height="50" uuid="..." />
  <c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components">
    <!-- contenido de la tabla -->
  </c:table>
</componentElement>
```

Línea 1	<code><componentElement></code> → declara un componente dentro de una banda del informe. El elemento <code>table</code> es uno de los tipos de componente disponibles.
Línea 2	<code><reportElement x="0" y="100" width="555" height="50" uuid="..." /></code> → posición y tamaño inicial del componente. La altura es la mínima y se amplía automáticamente según el número de filas.
Línea 3	<code><c:table xmlns:c="http://jasperreports.sourceforge.net/jasperreports/components"></code> → declara el elemento tabla. El prefijo <code>jr</code> se utiliza habitualmente para los componentes de JasperReports.

El elemento `table` tiene un namespace propio (`http://jasperreports.sourceforge.net/jasperreports/components`) que se declara con el prefijo `jr`. Dentro de este elemento se definen las columnas, el encabezado, las celdas de detalle y el dataset asociado. La tabla puede colocarse en cualquier banda del informe y se emite en el momento de la emisión de esa banda. Si la banda se emite varias veces, la tabla se ejecuta varias veces, una por cada emisión. La tabla es un componente autocontenido que no comparte el dataset del informe principal.

TEXT

ESTRUCTURA DE UNA TABLA



Qué representa el diagrama: la estructura jerárquica del elemento `table`. El `datasetRun` define cómo se alimenta la tabla. Cada `column` define una columna con su encabezado y su celda de detalle.

Por qué es relevante: permite comprender la organización interna del elemento y localizar cada parte de la definición.

Bloque 2 — El dataset de la tabla

La tabla se alimenta de un dataset propio que se declara en el nivel del informe con el elemento `dataset`. El dataset contiene su propia consulta SQL, sus propios campos y sus propios parámetros. La tabla hace referencia al dataset mediante el elemento

`datasetRun` que contiene la conexión o la fuente de datos. Cuando el motor emite la tabla, ejecuta la consulta del dataset con la conexión proporcionada y procesa los resultados en el contexto de la tabla. Esta separación entre el dataset del informe principal y el dataset de la tabla es la que permite que la tabla muestre datos distintos a los de la banda que la contiene.

XML

```
<subDataset name="DatasetVentasDetalle">
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql">
    <![CDATA[
      SELECT fecha_venta, cantidad, precio_unitario
      FROM ventas
      WHERE titulo_libro = $P{tituloLibro}
      ORDER BY fecha_venta
    ]]>
  </queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
```

Línea 1	<code><subDataset name="DatasetVentasDetalle"></code> → declara un subdataset con nombre identificable. El prefijo <code>sub</code> indica que es un dataset auxiliar del informe.
Línea 2	<code><parameter name="tituloLibro" class="java.lang.String"/></code> → declara el parámetro del dataset. Es independiente de los parámetros del informe principal.
Línea 3-9	<code><queryString></code> con la consulta SQL que recupera las ventas del libro.
Línea 10-12	las declaraciones de los campos del dataset.

Un subdataset puede compartir parámetros con el informe principal si el nombre coincide. El paso de parámetros del informe principal al subdataset se realiza con el elemento `datasetParameter` dentro del `datasetRun` de la tabla. El valor de cada parámetro se calcula en el contexto de la banda que contiene la tabla y se pasa al subdataset en el momento de la emisión. La coherencia de nombres entre el parámetro del subdataset y el `datasetParameter` es condición necesaria para que el paso funcione.

TEXT

PASO DE PARÁMETROS DEL INFORME AL SUBDATASET

```
Informe principal:
<subreportParameter name="tituloLibro">
  <subreportParameterExpression>$F{titulo}</subreportParameterExpression>
</subreportParameter>

Subdataset de la tabla:
<parameter name="tituloLibro" class="java.lang.String"/>

El datasetRun de la tabla conecta los dos:
<datasetParameter name="tituloLibro">
  <datasetParameterExpression>$F{titulo}</datasetParameterExpression>
</datasetParameter>
```

Qué representa el diagrama: el paso de parámetros del informe principal al subdataset de la tabla. La conexión se realiza en el `datasetRun`.

Por qué es relevante: permite que la tabla reciba los valores del registro actual del informe principal sin compartir el dataset completo.

Bloque 3 — Las columnas de la tabla

Cada columna de la tabla se declara con el elemento `column` que contiene un atributo `width` con el ancho en píxeles. Dentro de la columna se declaran dos secciones: `columnHeader` con los elementos que se emiten en el encabezado de la columna y `detailCell` con los elementos que se emiten una vez por cada registro del dataset. Cada columna puede tener su propio estilo y su propio ancho. El ancho total de las columnas debe coincidir con el ancho del elemento `table` para que la tabla se muestre correctamente alineada.

XML

```
<c:column width="150">
  <c:columnHeader height="20" rowSpan="1">
    <staticText>
      <reportElement x="0" y="0" width="150" height="20" uuid="..." />
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="9" isBold="true"/>
      </textElement>
      <text><![CDATA[Fecha]]></text>
    </staticText>
  </c:columnHeader>
  <c:detailCell height="15">
    <textField>
      <reportElement x="0" y="0" width="150" height="15" uuid="..." />
```

```

<textElement verticalAlignment="Middle">
  <font fontName="DejaVu Sans" size="9"/>
</textElement>
<textFieldExpression><![CDATA[{F{fecha_venta}}]></textFieldExpression>
</textField>
</c:detailCell>
</c:column>

```

Línea 1	<code><c:column width="150"></code> → declara una columna con 150 píxeles de ancho.
Línea 2	<code><c:columnHeader height="20" rowspan="1"></code> → declara el encabezado de la columna con 20 píxeles de altura. El atributo <code>rowSpan</code> permite que el encabezado ocupe varias filas cuando la tabla tiene más de un nivel de encabezado.
Línea 3-10	el <code>staticText</code> que se emite en el encabezado. La posición y el tamaño del elemento son relativos a la celda del encabezado.
Línea 11	<code></c:columnHeader></code> → cierra el encabezado.
Línea 12	<code><c:detailCell height="15"></code> → declara la celda de detalle con 15 píxeles de altura.
Línea 13-20	el <code>textField</code> que se emite en cada fila del detalle.
Línea 21	<code></c:detailCell></code> → cierra la celda de detalle.
Línea 22	<code></c:column></code> → cierra la columna.

La tabla emite una fila por cada registro del dataset. El motor recorre el dataset, emite la `detailCell` de cada columna para cada registro y construye la tabla fila a fila. El encabezado se emite una sola vez al inicio de la tabla, antes del detalle. Si la tabla no cabe completa en la página, el motor emite el encabezado de nuevo en la página siguiente para que el lector pueda identificar las columnas. Este comportamiento es el que hace que la tabla sea adecuada para conjuntos de datos con un número variable de filas.

TEXT

EMISIÓN DE LA TABLA

Encabezado (una vez por página de la tabla):

Fecha	Cantidad	Precio
-------	----------	--------

Fila 1:

2026-09-01	3	19,95 €
------------	---	---------

Fila 2:

2026-09-05	5	19,95 €
------------	---	---------

... (una fila por cada registro del dataset)

La tabla se amplía verticalmente según el número de filas.
Si la tabla no cabe en la página, continúa en la siguiente.

Qué representa el diagrama: la emisión de la tabla fila a fila. El encabezado se emite una vez por página. Cada registro del dataset produce una fila.

Por qué es relevante: permite comprender el comportamiento de la tabla ante conjuntos de datos de tamaño variable.

Bloque 4 — Estilos de la tabla

En JasperReports 6.20.0 una tabla no necesita un elemento especial `tableStyle`. Los estilos siguen siendo estilos JasperReports normales declarados en el informe o importados desde una plantilla. Esos estilos se asignan a `c:columnHeader`, `c:detailCell` o a los elementos que contienen. En EditorialReports se usan `M5TableHeader` y `M5TableDetail`; esta elección hace que la tabla participe del mismo sistema de estilos que el resto del informe.

XML

```

<style name="M5TableHeader" style="Dato" mode="Opaque"
  bgcolor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
<style name="M5TableDetail" style="Dato"/>
...
<c:columnHeader style="M5TableHeader" height="20">...</c:columnHeader>
<c:detailCell style="M5TableDetail" height="18">...</c:detailCell>

```

Lectura del ejemplo: las dos primeras líneas crean estilos de informe reutilizables; `style="Dato"` conserva la tipografía heredada; `mode="Opaque"` permite pintar el fondo del encabezado; las dos últimas líneas aplican los estilos a celdas reales de la tabla. Esta sintaxis se corresponde con el componente que compila el checkpoint 5.2.

Bloque 5 — Compilación de la tabla

La tabla es un componente interno del JRXML principal. Al compilar `informe_ventas.jrxml`, JasperReports genera **informe_ventas.jasper** y dentro de ese objeto compilado queda incluida la definición de la tabla y su `datasetRun`. No se genera un archivo independiente con sufijo `_table_1.jasper`. Esta corrección es importante porque la fuente original confundía la compilación del informe con la existencia de plantillas auxiliares separadas.

JAVA

```
JasperCompileManager.compileReportToFile(
    "reports/informe_ventas.jrxml",
    "reports/informe_ventas.jasper");
```

Después, `JasperFillManager.fillReport(...)` ejecuta el informe principal y, cuando alcanza la tabla, ejecuta su subdataset con la conexión indicada. El E2E de M5 verifica deliberadamente que `informe_ventas.jasper` existe y que no aparecen falsos artefactos `informe_ventas_table_*.jasper`.

Contrato ejecutable de DatasetTopVentas

La tabla del checkpoint 5.2 no recorre el dataset principal. Usa el subdataset real `DatasetTopVentas`, que recibe el parámetro `tituloLibro` y devuelve exactamente tres fields: `fecha_venta`, `cantidad` y `precio_unitario`.

XML

```
<subDataset name="DatasetTopVentas">
  <parameter name="tituloLibro" class="java.lang.String"/>
  <queryString language="sql"><![CDATA[
    SELECT fecha_venta, cantidad, precio_unitario
    FROM ventas
    WHERE titulo_libro = ${tituloLibro}
    ORDER BY cantidad DESC, fecha_venta
    LIMIT 3
  ]]></queryString>
  <field name="fecha_venta" class="java.lang.String"/>
  <field name="cantidad" class="java.lang.Integer"/>
  <field name="precio_unitario" class="java.lang.Double"/>
</subDataset>
```

El `datasetRun` de `c:table` pasa `${titulo}` a `tituloLibro` y reutiliza `${REPORT_CONNECTION}`. Por tanto, cada fila del informe maestro obtiene su propio `Top 3` sin alterar la consulta principal. Los encabezados usan `M5TableHeader` y las celdas de detalle `M5TableDetail`.

Este contrato conecta directamente la teoría con la Parte A, el JRXML completo de Parte B y la ejecución de Parte C: si cambia el nombre del subdataset, del parámetro o de cualquiera de los fields, la tabla deja de reproducir el checkpoint validado.

Resumen rápido de la teoría

- La tabla es un componente que organiza datos en filas y columnas.
- Se declara con el elemento `componentElement` y el elemento `c:table`.
- La tabla se alimenta de un `subDataset` con su propia consulta y sus propios campos.
- El `datasetRun` conecta el subdataset con la conexión o la fuente de datos.
- Los parámetros se pasan con `datasetParameter`.
- Cada columna se declara con `jr:column` y contiene un `columnHeader` y un `detailCell`.
- Los estilos son estilos JasperReports normales aplicados a `c:columnHeader` y `c:detailCell`.
- La tabla se compila dentro de `informe_ventas.jasper`; no genera un `.jasper` independiente.

- La tabla ejecutable usa `DatasetTopVentas`, recibe `tituloLibro` y presenta `fecha_venta`, `cantidad` y `precio_unitario`.

Punto 5.3 — Agrupaciones

Objetivos de aprendizaje

- Comprender el elemento `group` y su papel en la organización de los registros.
- Declarar grupos en el JRXML con su expresión de agrupación.
- Configurar las bandas `groupHeader` y `groupFooter` de cada grupo.
- Declarar variables con `resetType="Group"` para calcular subtotales por grupo.
- Utilizar las propiedades `isStartNewPage`, `isReprintHeaderOnEachPage` y `minHeightToStartNewPage`.
- Documentar las agrupaciones del proyecto EditorialReports.

Parte teórica

Bloque 1 — El elemento `group` y su papel en la organización

Un grupo en JasperReports es una sección del informe que se emite cada vez que cambia el valor de una expresión. La expresión se denomina expresión de agrupación y se declara en el elemento `groupExpression`. El motor evalúa la expresión en cada registro del informe y, cuando el valor cambia, cierra el grupo actual y abre uno nuevo. Este comportamiento permite organizar los registros por categoría, por año, por rango de precio o por cualquier otro criterio. La agrupación es la técnica que permite construir informes con secciones que se repiten un número indeterminado de veces según los datos.

```
<group name="CategoriaGroup">
  <groupExpression><![CDATA[{$F{categoria}}]></groupExpression>
  <groupHeader>
    <band height="20">
      <staticText>
        <reportElement x="0" y="0" width="555" height="20" uuid="..." />
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="11" isBold="true" />
          </textElement>
          <text><![CDATA[Categoría:]]></text>
        </staticText>
      </band>
    </groupHeader>
  </group>
```

XML

Línea 1	<code><group name="CategoriaGroup"></code> → declara un grupo con nombre identificable. El nombre se utiliza para referenciar el grupo desde las variables y desde otras partes del informe.
Línea 2	<code><groupExpression><![CDATA[{\$F{categoria}}]></groupExpression></code> → expresión de agrupación. El motor evalúa el campo <code>categoria</code> en cada registro y agrupa los registros consecutivos que tienen el mismo valor.
Línea 3-13	<code><groupHeader></code> → banda que se emite al inicio de cada grupo. En este caso contiene un texto estático con la etiqueta <code>Categoría:</code> . La banda del encabezado puede incluir también el valor del campo de agrupación mediante una expresión.

Los grupos se declaran dentro del elemento raíz `jasperReport` y antes de las bandas del informe. Un informe puede contener varios grupos anidados. El orden de declaración determina la jerarquía: el primer grupo es el más externo y el último es el más interno. Cuando un registro cambia el valor de la expresión del grupo externo, el motor cierra todos los grupos internos antes de cerrar el externo. Esta jerarquía permite construir informes con agrupaciones anidadas, como un informe de ventas agrupado por categoría y dentro de cada categoría por año.

```
JERARQUÍA DE GRUPOS ANIDADOS

Grupo externo: Categoría = "Novela"
├── Grupo interno: Año = 1967
│   ├── Registro: Cien años de soledad
│   └── (fin del grupo Año 1967)
├── Grupo interno: Año = 1963
│   ├── Registro: Rayuela
│   ├── Registro: La ciudad y los perros
│   └── (fin del grupo Año 1963)
└── (fin del grupo Categoría "Novela")

Grupo externo: Categoría = "Ensayo"
```

TEXT

L ...

Qué representa el diagrama: la jerarquía de grupos anidados. El grupo externo agrupa por categoría y el grupo interno agrupa por año dentro de cada categoría.

Por qué es relevante: permite construir informes con múltiples niveles de agrupación sin escribir código procedural.

Bloque 2 — Las bandas groupHeader y groupFooter

Cada grupo tiene dos bandas asociadas: groupHeader y groupFooter. La banda groupHeader se emite al inicio de cada grupo, antes del primer registro del grupo. La banda groupFooter se emite al final de cada grupo, después del último registro. Ambas bandas son opcionales: un grupo puede tener solo el encabezado, solo el pie o ambos. La banda groupHeader se utiliza habitualmente para mostrar el valor de la agrupación y los encabezados de columna del grupo. La banda groupFooter se utiliza para mostrar los subtotales del grupo.

XML

```
<group name="CategoriaGroup">
  <groupExpression><![CDATA[{F{categoria}}]></groupExpression>
  <groupHeader>
    <band height="20">
      <textField>
        <reportElement x="0" y="0" width="300" height="20" uuid="..." />
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="11" isBold="true"/>
        </textElement>
        <textFieldExpression><![CDATA["Categoría: " + {F{categoria}}]></textFieldExpression>
      </textField>
    </band>
  </groupHeader>
  <groupFooter>
    <band height="20">
      <staticText>
        <reportElement x="0" y="0" width="200" height="20" uuid="..." />
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Subtotal categoría: ]></text>
      </staticText>
      <textField pattern="#,##0.00 €">
        <reportElement x="200" y="0" width="130" height="20" uuid="..." />
        <textElement textAlignment="Right" verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <textFieldExpression><![CDATA[{V{GrupoImporte}}]></textFieldExpression>
      </textField>
    </band>
  </groupFooter>
</group>
```

Línea 1	<code><group name="CategoriaGroup"></code> → declara el grupo.
Línea 2	<code><groupExpression><![CDATA[{F{categoria}}]></groupExpression></code> → expresión de agrupación por categoría.
Línea 3-13	<code><groupHeader></code> con la banda de encabezado. La banda contiene un <code>textField</code> que muestra el valor de la categoría.
Línea 14-34	<code><groupFooter></code> con la banda de pie. La banda contiene un <code>staticText</code> con el rótulo <code>Subtotal categoría:</code> y un <code>textField</code> con la variable <code>GrupoImporte</code> .

La banda groupHeader se emite al inicio de cada grupo. Si el grupo tiene muchos registros y ocupa varias páginas, la banda groupHeader se emite una sola vez, al principio del grupo. El atributo `isReprintHeaderOnEachPage` de la banda permite que el encabezado se reimprima en cada página del grupo para que el lector pueda identificar la categoría. La banda groupFooter se emite una sola vez, al final del grupo. Si el grupo tiene un pie y el grupo termina al final de una página, el pie se emite antes del salto de página.

TEXT

EMISIÓN DE LAS BANDAS DEL GRUPO

Grupo "Novela":

groupHeader: "Categoría..."	← al inicio del grupo
Detail: Cien años de sol.	
Detail: Rayuela	
Detail: La ciudad...	
Detail: Pedro Páramo	
Detail: Ficciones	

groupFooter: "Subtotal..."	← al final del grupo
----------------------------	----------------------

Grupo "Ensayo":

groupHeader: "Categoría..."
Detail: ...
groupFooter: "Subtotal..."

Qué representa el diagrama: la emisión de las bandas del grupo. El encabezado se emite al inicio, el pie al final. La banda detail se emite para cada registro del grupo.

Por qué es relevante: permite comprender el orden de emisión de las bandas y el comportamiento del grupo cuando ocupa varias páginas.

Bloque 3 — Variables con resetType="Group"

Una variable con `resetType="Group"` se reinicia al inicio de cada grupo. La combinación del cálculo y del reinicio permite calcular subtotales por grupo. El atributo `resetGroup` de la variable indica el nombre del grupo que dispara el reinicio. El motor reinicia la variable cada vez que se abre un nuevo grupo con ese nombre. La variable acumula el valor a lo largo del grupo y se reinicia al inicio del siguiente. La banda `groupFooter` es el lugar natural para mostrar el valor final del subtotal antes de que la variable se reinicie.

XML

```
<variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[{$importe_total}]]></variableExpression>
</variable>
<variable name="GrupoLibros" class="java.lang.Integer" calculation="Count" resetType="Group" resetGroup="CategoriaGroup">
  <variableExpression><![CDATA[{$titulo}]]></variableExpression>
</variable>
```

Línea 1	<code><variable name="GrupoImporte" class="java.lang.Double" calculation="Sum" resetType="Group" resetGroup="CategoriaGroup"></code> → declara una variable que acumula el importe total mediante suma y se reinicia al inicio de cada grupo <code>CategoriaGroup</code> .
Línea 2	<code><variableExpression><![CDATA[{\$importe_total}]]></variableExpression></code> → expresión que se evalúa en cada registro del grupo.
Línea 4	<code><variable name="GrupoLibros" ...></code> → declara una variable que cuenta los libros de cada categoría con reinicio por grupo.

La declaración de la variable con `resetType="Group"` requiere que el grupo exista y que el atributo `resetGroup` coincida con el nombre del grupo. Si el grupo no existe o el nombre no coincide, el compilador lanza un error. La variable se declara antes del grupo en el JRXML porque el atributo `resetGroup` la referencia. La organización del archivo es: parámetros, campos, variables, grupos y bandas. El motor procesa las variables antes de los grupos y las reinicia cuando el grupo correspondiente se abre. La coherencia entre el nombre de la variable, el nombre del grupo y el atributo `resetGroup` es condición necesaria para que el subtotal se calcule correctamente.

TEXT

CICLO DE VIDA DE UNA VARIABLE CON RESETTYPE="GROUP"

```
Inicio del grupo "Novela":
  GrupoLibros = 0
  GrupoImporte = 0.0
  |
  ▼
Registro 1: Cien años de soledad
  GrupoLibros = 1
  GrupoImporte = 159.60

Registro 2: Rayuela
  GrupoLibros = 2
  GrupoImporte = 294.60

...

Último registro del grupo: Paradiso
  GrupoLibros = 12
  GrupoImporte = 252.55

Fin del grupo "Novela" → se emite la banda groupFooter con los valores finales.
|
▼
Inicio del grupo "Ensayo" → las variables se reinician.
```

Qué representa el diagrama: el ciclo de vida de una variable con reinicio por grupo. La variable acumula durante el grupo y se reinicia al inicio del siguiente.

Por qué es relevante: permite comprender por qué el subtotal se muestra en la banda `groupFooter` con el valor acumulado del grupo.

Bloque 4 — Propiedades del elemento group

El elemento `group` admite varias propiedades que controlan su comportamiento. El atributo `isStartNewPage` determina si el grupo debe comenzar en una página nueva. El atributo `isReprintHeaderOnEachPage` determina si el encabezado del grupo se debe reimprimir en cada página del grupo. El atributo `minHeightToStartNewPage` define la altura mínima que debe quedar al final de la página para que el grupo pueda comenzar en ella. Si el espacio disponible es inferior a este valor, el motor emite un salto de página antes de comenzar el grupo. Estas propiedades permiten controlar el comportamiento del grupo en relación con los saltos de página.

XML

```
<group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80">
  <groupExpression><![CDATA[{$F{categoria}}]></groupExpression>
  <groupHeader>
    <band height="25">
      ...
    </band>
  </groupHeader>
  <groupFooter>
    <band height="25">
      ...
    </band>
  </groupFooter>
</group>
```

Línea 1

`<group name="CategoriaGroup" isStartNewPage="false" isReprintHeaderOnEachPage="true" minHeightToStartNewPage="80">` → declara el grupo con tres propiedades. `isStartNewPage="false"` hace que cada grupo comience en una página nueva. `isReprintHeaderOnEachPage="true"` reimprime el encabezado del grupo en cada página. `minHeightToStartNewPage="80"` exige al menos 60 píxeles libres al final de la página para que el grupo pueda comenzar.

La propiedad `isStartNewPage` resulta útil cuando cada grupo debe ocupar una sección independiente del documento. Un informe de facturas con una factura por grupo puede comenzar cada factura en una página nueva. La propiedad `isReprintHeaderOnEachPage` resulta útil cuando el grupo ocupa varias páginas y el lector necesita identificar la categoría en cada página. La propiedad `minHeightToStartNewPage` resulta útil cuando el encabezado y el primer registro del grupo deben aparecer juntos en la misma página. La combinación de las tres propiedades permite construir informes visualmente coherentes y evitar saltos de página que separan el encabezado de su contenido.

TEXT

COMPORTAMIENTO DE LAS PROPIEDADES DEL GRUPO

```
isStartNewPage="false" (por defecto):
  El grupo comienza en la misma página que el anterior si hay espacio.
isStartNewPage="true":
  El grupo comienza en una página nueva siempre.

isReprintHeaderOnEachPage="false" (por defecto):
  El encabezado del grupo se emite solo al inicio del grupo.
isReprintHeaderOnEachPage="true":
  El encabezado del grupo se reimprime al inicio de cada página del grupo.

minHeightToStartNewPage="0" (por defecto):
  El grupo comienza en la página actual si hay espacio, aunque sea mínimo.
minHeightToStartNewPage="80":
  El grupo comienza en la página actual solo si hay al menos 60 píxeles libres.
```

Qué representa el diagrama: el comportamiento de las propiedades del grupo. Cada propiedad controla un aspecto distinto del comportamiento ante los saltos de página.

Por qué es relevante: permite controlar la distribución del grupo en el documento y evitar efectos visuales indeseados.

Bloque 5 — Combinación de grupos con el subdataset y las tablas

Los grupos se combinan con los subreportes y las tablas para construir informes con múltiples niveles de detalle. Un informe puede tener un grupo por categoría, y dentro de cada grupo, una tabla con los libros de esa categoría. El grupo organiza los registros del informe principal y la tabla muestra los datos relacionados del subdataset. La combinación de ambos niveles permite construir informes jerárquicos que no pueden representarse con una sola consulta SQL. La coordinación entre los dos niveles se realiza mediante el paso de parámetros del grupo a la tabla.

XML

```

<group name="CategoriaGroup">
  <groupExpression><![CDATA[{$F{categoria}}]></groupExpression>
  <groupHeader>
    <band height="25">
      <textField>
        <reportElement x="0" y="0" width="300" height="20" uuid="..." />
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="11" isBold="true" />
        </textElement>
        <textFieldExpression><![CDATA["Categoría: " + {$F{categoria}}]></textFieldExpression>
      </textField>
    </band>
  </groupHeader>
  <detail>
    <band height="30">
      <textField>
        <reportElement x="0" y="0" width="555" height="20" uuid="..." />
        <textFieldExpression><![CDATA[{$F{titulo}}]></textFieldExpression>
      </textField>
    </band>
  </detail>
  <groupFooter>
    <band height="20">
      <staticText>
        <reportElement x="0" y="0" width="200" height="20" uuid="..." />
        <text><![CDATA[Total categoría: ]]></text>
      </staticText>
      <textField pattern="#,##0.00 €">
        <reportElement x="200" y="0" width="130" height="20" uuid="..." />
        <textFieldExpression><![CDATA[{$V{GrupoImporte}}]></textFieldExpression>
      </textField>
    </band>
  </groupFooter>
</group>

```

Línea 1	<group name="CategoriaGroup"> → declara el grupo por categoría.
Línea 3-15	<groupHeader> con el encabezado del grupo que muestra la categoría.
Línea 16-23	<detail> con la banda de detalle del grupo. Cada libro de la categoría se emite en esta banda.
Línea 24-38	<groupFooter> con el subtotal de la categoría.

La combinación de grupos con tablas permite construir informes en los que cada grupo contiene una tabla con datos adicionales. El grupo muestra la información agregada y la tabla muestra el detalle. Esta estructura es habitual en informes financieros, en informes de ventas y en informes de inventario. La coordinación entre el grupo y la tabla se realiza mediante el paso del valor de agrupación como parámetro del datasetRun. La tabla puede tener su propia conexión o recibir la conexión del informe mediante el parámetro interno REPORT_CONNECTION. La combinación de los dos niveles construye un documento con la estructura jerárquica adecuada para el lector.

TEXT

ESTRUCTURA CON GRUPOS Y TABLAS

Grupo "Novela":

groupHeader: "Categoría: Novela"
Detail: Cien años de soledad Detail: Rayuela ...
groupFooter: "Total: 159,60 €"

Grupo "Ensayo":

groupHeader: "Categoría: Ensayo"
Detail: ...
groupFooter: "Total: ..."

Qué representa el diagrama: la estructura del informe con grupos por categoría. Cada grupo contiene sus registros de detalle y su subtotal.

Por qué es relevante: permite construir informes con estructura jerárquica que organizan los datos en secciones claramente identificables.

Resumen rápido de la teoría

- Un grupo es una sección del informe que se emite cada vez que cambia una expresión.
- El elemento `group` contiene `groupExpression`, `groupHeader` y `groupFooter`.
- Las bandas `groupHeader` y `groupFooter` son opcionales.
- Las variables con `resetType="Group"` calculan subtotales por grupo.
- El atributo `resetGroup` de la variable indica el grupo que dispara el reinicio.
- Las propiedades del grupo controlan los saltos de página y la reimpresión del encabezado.
- Los grupos se pueden anidar para construir jerarquías de varios niveles.
- Los grupos se combinan con tablas y subreportes para construir informes complejos.

Punto 5.4 — Gráficos

Objetivos de aprendizaje

- Comprender el elemento `chart` y sus componentes internos.
- Declarar un subdataset propio para alimentar el gráfico.
- Configurar los ejes de categorías y de valores del gráfico.
- Elegir el tipo de gráfico adecuado según la naturaleza de los datos.
- Aplicar estilos y títulos al gráfico y a sus series.
- Documentar los gráficos del proyecto EditorialReports.

Parte teórica

Bloque 1 — El gráfico en JasperReports 6.20.0 y su ciclo de ejecución

Un gráfico de JasperReports no es una imagen decorativa que se añade después de generar el informe. Forma parte del árbol JRXML, se compila junto con el resto del documento y se alimenta durante la fase de `fill`. En JasperReports Library 6.20.0 los gráficos clásicos se representan mediante elementos nativos como `barChart`, `pieChart` o `lineChart`. En el checkpoint 5.4 de EditorialReports se usa un `barChart` colocado directamente dentro de la banda `Summary`. No se envuelve en un `componentElement` y tampoco utiliza un namespace inventado `chart:`.

El bloque `chart` concentra las propiedades comunes: posición y tamaño a través de `reportElement`, título, subtítulo y leyenda. Después aparecen el dataset específico del tipo de gráfico y su plot.

```
<barChart>
  <chart>
    <reportElement x="0" y="165" width="555" height="250"/>
    <chartTitle>
      <titleExpression><![CDATA["Ventas por categoría"]]></titleExpression>
    </chartTitle>
    <chartSubTitle/>
    <chartLegend position="Bottom"/>
  </chart>
  ...
</barChart>
```

XML

Línea 1	<code><barChart></code> → abre el gráfico de barras nativo que JasperReports compilará dentro del informe maestro.
Línea 3	<code><reportElement x="0" y="165" width="555" height="250"/></code> → fija exactamente la geometría validada del gráfico en <code>Summary</code> .
Líneas 4-6	<code>chartTitle</code> y <code>titleExpression</code> → definen el título visible sin depender de un elemento de texto externo.
Línea 8	<code><chartLegend position="Bottom"/></code> → coloca la leyenda debajo del área de trazado.

Durante la compilación, `JasperCompileManager` transforma esta definición en objetos internos del `informe_ventas.jasper`. Durante el llenado, JasperReports ejecuta el dataset del gráfico, construye las categorías y valores y finalmente pinta el componente en el `JasperPrint`. Por eso una definición que sea XML válido pero use elementos equivocados puede fallar al compilar, y una definición que compile pero use un dataset incorrecto puede producir un gráfico vacío o semánticamente erróneo.

Qué representa el diagrama: el flujo real del gráfico desde JRXML hasta PDF.

```
barChart en JRXML
├── chart: geometría + título + leyenda
├── categoryDataset: datos y serie
└── barPlot: ejes y representación
    ▼
informe_ventas.jasper
    ▼
JasperFillManager + SQLite
    ▼
JasperPrint -> informe_ventas.pdf
```

TEXT

Por qué es relevante: evita confundir el gráfico con un recurso externo. En este módulo el gráfico no genera ningún `_chart_1.jasper` independiente.

Bloque 2 — Dataset independiente y consulta agregada por categoría

El informe principal necesita conservar una fila por libro para seguir mostrando los catorce títulos del catálogo incluso cuando un libro no tenga ventas. El gráfico, en cambio, necesita una fila por categoría. Mezclar ambos objetivos en la misma consulta obligaría a cambiar la granularidad del dataset principal y rompería la maquetación acumulada. La solución correcta es declarar `DatasetVentasPorCategoria` como `subDataset`.

El checkpoint 5.4 usa una consulta que parte de `libros` y realiza un `LEFT JOIN` con `ventas`. La función `COALESCE` convierte en 0,0 el importe de una categoría si la suma pudiera resultar nula.

XMI

```
<subDataset name="DatasetVentasPorCategoria">
  <queryString language="sql">
    <![CDATA[
      SELECT l.categoria AS categoria_grafico,
             COALESCE(SUM(v.cantidad * v.precio_unitario), 0.0) AS importe_categoria
      FROM libros l
      LEFT JOIN ventas v ON l.titulo = v.titulo_libro
      GROUP BY l.categoria
      ORDER BY l.categoria
    ]]>
  </queryString>
  <field name="categoria_grafico" class="java.lang.String"/>
  <field name="importe_categoria" class="java.lang.Double"/>
</subDataset>
```

Línea 1	<code>DatasetVentasPorCategoria</code> → crea un ámbito de datos independiente del dataset principal.
Línea 4	<code>1.categoria AS categoria_grafico</code> → expone la dimensión categórica con un nombre que coincide con el field JRXML.
Línea 5	<code>COALESCE(SUM(...), 0.0) AS importe_categoria</code> → calcula el importe agregado y garantiza un valor numérico utilizable por el gráfico.
Línea 7	<code>LEFT JOIN ventas...</code> → mantiene la categoría aunque alguno de sus libros no tenga movimientos de venta.
Línea 8	<code>GROUP BY 1.categoria</code> → reduce todas las ventas de una categoría a una sola fila agregada.

El contrato entre SQL y JRXML es estricto. Si la consulta devuelve `importe_categoria` pero el field se llama `importe_categoria`, JasperReports no puede resolver la expresión del gráfico. Por eso la teoría, la práctica visual y el checkpoint usan exactamente los mismos nombres.

Qué representa el diagrama: dos granularidades que coexisten en el mismo informe.

TEXT

Dataset principal	DatasetVentasPorCategoria
1 fila por libro	1 fila por categoría
14 títulos	categorías del catálogo
campos de detalle	categoría_grafico
totales del informe	importe_categoria
└─ mantiene fichas	└─ alimenta barChart

Por qué es relevante: los subdatasets permiten añadir análisis sin alterar la consulta que sustenta la parte principal del documento.

Bloque 3 — Elegir el tipo de gráfico según la pregunta analítica

El tipo de gráfico debe elegirse a partir de la relación que se quiere comunicar, no por preferencia estética. En 5.4 la pregunta es: “¿qué importe de ventas corresponde a cada categoría editorial?”. El eje horizontal contiene categorías discretas y el eje vertical una magnitud comparable. Un gráfico de barras es adecuado porque permite comparar longitudes con una escala común.

TEXT

Pregunta analítica	Tipo habitual
Comparar magnitudes entre categorías	barChart
Mostrar composición de un total	pieChart
Mostrar evolución ordenada en el tiempo	lineChart / timeSeriesChart
Relacionar dos magnitudes numéricas	xylineChart / scatterChart

Un pieChart podría representar el porcentaje de cada categoría sobre el total, pero dificultaría comparar valores próximos y no mostraría con la misma claridad el importe absoluto. Un lineChart sugeriría continuidad u orden temporal entre categorías que no existe. Un gráfico XY requeriría dos ejes numéricos, algo que tampoco corresponde a este conjunto de datos.

La elección del gráfico afecta también al dataset. `barChart` usa un `categoryDataset`; dentro de él cada `categorySeries` necesita una expresión de serie, otra de categoría y otra de valor. Si se cambia el tipo de gráfico hay que revisar la estructura JRXML asociada y no limitarse a cambiar el nombre de la etiqueta externa.

En EditorialReports se usa una única serie lógica llamada "Importe". Esto significa que la leyenda no distingue múltiples métricas; identifica una sola medida que se repite para cada categoría. La categoría se obtiene de `{categoria_grafico}` y la altura de cada barra de `{importe_categoria}`.

Qué representa la decisión: una correspondencia entre pregunta, estructura de datos y codificación visual.

TEXT

```

categoria editorial → posición en eje X
importe de ventas  → altura de la barra
"Importe"          → nombre de serie / leyenda

```

Por qué es relevante: un informe puede compilar y ser técnicamente correcto pero comunicar mal si el tipo de gráfico no corresponde a la naturaleza de los datos. La validación de 5.4 es, por tanto, sintáctica, ejecutable y semántica.

Bloque 4 — DatasetRun, serie, categoría y valor

El `categoryDataset` conecta el gráfico con `DatasetVentasPorCategoria`. La conexión se realiza mediante `datasetRun`. Al igual que en la tabla del punto 5.2, se reutiliza `{REPORT_CONNECTION}` para que el subdataset ejecute su SQL con la misma conexión JDBC que ya usa el informe maestro.

XML

```

<categoryDataset>
  <dataset>
    <datasetRun subDataset="DatasetVentasPorCategoria">
      <connectionExpression><![CDATA[{REPORT_CONNECTION}]]></connectionExpression>
    </datasetRun>
  </dataset>
  <categorySeries>
    <seriesExpression><![CDATA["Importe"]]></seriesExpression>
    <categoryExpression><![CDATA[{categoria_grafico}]]></categoryExpression>
    <valueExpression><![CDATA[{importe_categoria}]]></valueExpression>
  </categorySeries>
</categoryDataset>

```

Línea 1	<code>categoryDataset</code> → abre la estructura de datos propia de un gráfico categórico.
Línea 3	<code>datasetRun subDataset="DatasetVentasPorCategoria"</code> → selecciona el subdataset que debe ejecutarse.
Línea 4	<code>REPORT_CONNECTION</code> → reutiliza la conexión JDBC existente y evita abrir otra conexión desde Java.
Línea 8	<code>seriesExpression</code> → asigna el nombre Importe a la serie.
Línea 9	<code>categoryExpression</code> → toma la categoría de cada fila devuelta por el subdataset.
Línea 10	<code>valueExpression</code> → toma el importe agregado que determina la longitud de cada barra.

Los fields usados aquí pertenecen al contexto de `DatasetVentasPorCategoria`, no al dataset principal. Esa diferencia de ámbito es fundamental. Dentro de `categorySeries`, `{categoria_grafico}` y `{importe_categoria}` existen porque fueron declarados como fields del subdataset. Si se intentara usar un field exclusivo del dataset principal sin pasarlo o declararlo en el subdataset, la expresión sería inválida en ese contexto.

El mismo principio se aplica a parámetros: un subdataset no “hereda mágicamente” parámetros arbitrarios. En este caso no necesita parámetros propios, únicamente la conexión. Si en un ejercicio posterior se quisiera filtrar el gráfico por un parámetro independiente, habría que declarar ese parámetro en el subdataset y pasarlo mediante `datasetParameter`.

Qué representa el diagrama: la separación entre contexto principal y contexto del gráfico.

TEXT

```

REPORT_CONNECTION
  |
  v
datasetRun
  |
  v
DatasetVentasPorCategoria
  |
  +-- categoria_grafico → categoryExpression
  |
  +-- importe_categoria → valueExpression

```

Por qué es relevante: evita errores de scope y explica por qué el gráfico puede ejecutar una consulta diferente sin modificar el generador Java.

Bloque 5 — Summary, plot, ejes, título, compilación y evidencia E2E

El gráfico no se coloca de forma aislada: convive con el resumen acumulado del informe. En el checkpoint 5.4 la banda Summary tiene altura 430. Los elementos de resumen heredados ocupan la zona superior; en $y=140$ se añade el rótulo Ventas por categoría — importe y el barChart empieza en $y=165$ con tamaño 555×250 . Esta geometría evita solapes y deja el componente dentro de los límites de la banda.

El barPlot del checkpoint mantiene una configuración deliberadamente sencilla.

XML

```
<barPlot>
  <plot/>
  <itemLabel/>
  <categoryAxisFormat><axisFormat/></categoryAxisFormat>
  <valueAxisFormat><axisFormat/></valueAxisFormat>
</barPlot>
```

Línea 1	barPlot → abre las opciones específicas del gráfico de barras.
Línea 2	plot → conserva el plot base sin sobrescribir color, transparencia u orientación.
Línea 3	itemLabel → incluye el nodo de configuración de etiquetas de ítems.
Línea 4	categoryAxisFormat → mantiene el formato del eje de categorías.
Línea 5	valueAxisFormat → mantiene el formato del eje numérico.

No se introduce seriesColor directamente dentro de barPlot porque esa no es la estructura usada por el checkpoint. Tampoco se añade position="Top" a chartTitle. La práctica visual debe llevar exactamente al XML que se compila en la Parte B.

La compilación del informe maestro incorpora el gráfico en reports/informe_ventas.jasper. No se produce un .jasper auxiliar separado para el gráfico. El E2E verifica explícitamente la ausencia de ese falso artefacto, compila todos los JRXML acumulados, inicializa SQLite, llena el informe y comprueba que el PDF de 5.4 tiene seis páginas.

TEXT

```
Summary height = 430
├─ resumen heredado      y=5..121
├─ rótulo del gráfico    y=140, h=20
└─ barChart              y=165, h=250
    ├── DatasetVentasPorCategoría
    ├── categorySeries
    └─ barPlot
```

La evidencia de datos permanece inalterada: 14 libros, 9 ventas, 31 unidades y 633,40 €. Que aparezca el gráfico no autoriza a cambiar la consulta principal ni los invariantes del informe.

Qué representa el diagrama: la ubicación física y el contrato de ejecución del gráfico.

Por qué es relevante: un diseño, JRXML, compilación y resultado PDF en una misma cadena verificable. Ese es el criterio que permite considerar el punto 5.4 realmente terminado y no sólo visualmente “parecido”.

Resumen rápido de la teoría

- Los gráficos clásicos de JasperReports 6.20.0 usan elementos nativos como barChart.
- Un gráfico puede alimentarse de un subDataset mediante datasetRun.
- categorySeries define serie, categoría y valor.
- El tipo de gráfico debe corresponder a la pregunta analítica.
- Título, leyenda y plot se configuran dentro del gráfico.
- El gráfico queda integrado en informe_ventas.jasper; no genera un _chart_N.jasper separado.

Punto 5.5 — Crosstabs

Objetivos de aprendizaje

- Comprender el elemento `crosstab` y su estructura de filas, columnas y medidas.
- Declarar un subdataset propio para alimentar la tabla cruzada.
- Configurar los grupos de fila (`rowGroup`) y de columna (`columnGroup`).
- Definir la celda de medida (`measure`) con su cálculo y su formato.
- Aplicar estilos a las celdas del crosstab.
- Documentar las tablas cruzadas del proyecto EditorialReports.

Parte teórica

Bloque 1 — El elemento crosstab y su estructura

En JasperReports 6.20.0 el crosstab es un elemento nativo del JRXML. En el checkpoint 5.5 se coloca directamente dentro de la banda Summary; no se envuelve en un `componentElement`. El propio crosstab contiene su `reportElement`, el dataset, los grupos, las medidas y las celdas.

```
<crosstab>
  <reportElement x="0" y="455" width="555" height="225"/>
  <crosstabDataset>
    <dataset>
      <datasetRun subDataset="DatasetCrosstabVentas">
        <connectionExpression><![CDATA[{P{REPORT_CONNECTION}}]></connectionExpression>
      </datasetRun>
    </dataset>
  </crosstabDataset>
  ...
</crosstab>
```

XML

Línea 1	<code><crosstab></code> → abre la tabla cruzada nativa.
Línea 2	<code><reportElement ... /></code> → fija posición y tamaño dentro de Summary.
Líneas 3-9	<code>crosstabDataset</code> y <code>datasetRun</code> → ejecutan <code>DatasetCrosstabVentas</code> con la conexión del informe principal.

El motor genera dinámicamente filas y columnas a partir de los grupos del crosstab y calcula las medidas en cada intersección. La definición completa se compila dentro de `informe_ventas.jasper`.

Bloque 2 — Los grupos de fila y de columna

Los grupos de fila y de columna se declaran con los elementos `rowGroup` y `columnGroup`. Cada grupo contiene un elemento `bucket` con un elemento `bucketExpression` que devuelve el valor de agrupación. El motor extrae los valores distintos de la expresión y los ordena según su orden natural. El atributo `width` del `rowGroup` define el ancho de la columna de etiquetas de fila. El atributo `height` del `columnGroup` define la altura de la fila de etiquetas de columna. El atributo `totalPosition` define la posición de la fila o columna de totales: Start, End, None.

```
<rowGroup name="CategoriaCross" width="150" totalPosition="End">
  <bucket>
    <bucketExpression><![CDATA[{F{categoria_cross}}]></bucketExpression>
  </bucket>
</rowGroup>
<columnGroup name="AnioCross" height="30" totalPosition="End">
  <bucket>
    <bucketExpression><![CDATA[{F{anio_cross}}]></bucketExpression>
  </bucket>
</columnGroup>
```

XML

Línea 1-5	<code><rowGroup name="CategoriaCross" width="150" totalPosition="End"></code> → declara el grupo de fila <code>CategoriaCross</code> con ancho 150 píxeles y total al final. La expresión <code>{F{categoria_cross}}</code> determina los valores distintos que aparecerán en las filas.
Línea 6-10	<code><columnGroup name="AnioCross" height="30" totalPosition="End"></code> → declara el grupo de columna <code>AnioCross</code> con altura 30 píxeles y total al final. La expresión <code>{F{anio_cross}}</code> determina los valores distintos que aparecerán en las columnas.

Los grupos pueden anidarse para construir matrices con varios niveles. Un crosstab puede tener dos grupos de fila (por ejemplo, categoría y subcategoría) y dos grupos de columna (por ejemplo, año y trimestre). La combinación de varios grupos produce una matriz con filas y columnas jerárquicas. El orden de declaración determina la jerarquía: el primer grupo es el más externo y el último es el más interno. La anidación de grupos es una de las características que hacen del crosstab una herramienta potente para el análisis multidimensional.

TEXT

CROSSTAB CON DOS GRUPOS DE FILA Y DOS DE COLUMNA

		2025			2026		
		T1	T2	T3	T1	T2	T3
Novela	Romance	...	...	...	...	...	...
	Histór.	...	...	...	...	...	...
Ensayo	Filosof.	...	...	...	...	...	...
	Científ.	...	...	...	...	...	...

Qué representa el diagrama: la estructura de un crosstab con dos grupos de fila anidados (categoría y subcategoría) y dos grupos de columna anidados (año y trimestre). La matriz tiene cuatro niveles.

Por qué es relevante: permite construir matrices multidimensionales que muestran las relaciones entre varias dimensiones de los datos.

Bloque 3 — Las medidas y sus cálculos

Una medida se declara con el elemento `measure` y define el valor que se muestra en las celdas del crosstab. El atributo `name` identifica la medida dentro del crosstab. El atributo `class` indica el tipo Java del valor. El atributo `calculation` indica el tipo de cálculo: `Sum`, `Count`, `Average`, `Lowest`, `Highest`, `StandardDeviation`, `Variance`, `First`, `DistinctCount`. El elemento hijo `measureExpression` contiene la expresión que devuelve el valor que se acumula. La medida se evalúa en el contexto de cada celda del crosstab, es decir, para cada combinación de fila y columna.

XML

```
<measure name="ImporteCross" class="java.lang.Double" calculation="Sum">
  <measureExpression><![CDATA[$F{importe_cross}]]></measureExpression>
</measure>
```

Línea 1 `<measure name="ImporteCross" class="java.lang.Double" calculation="Sum">` → declara la medida `ImporteCross` con tipo `Double` y cálculo `Sum`.

Línea 2 `<measureExpression><![CDATA[$F{importe_cross}]]></measureExpression>` → expresión que devuelve el valor que se acumula en cada celda.

Línea 3 `</measure>` → cierra la declaración de la medida.

Un crosstab puede tener varias medidas. Cada medida se muestra en una celda propia y su valor se calcula de forma independiente. Un crosstab de ventas puede mostrar el importe total con cálculo `Sum` y el número de ventas con cálculo `Count`. La combinación de varias medidas permite construir matrices que muestran distintas perspectivas del mismo conjunto de datos. La celda de cada medida puede tener su propio formato, su propio estilo y su propia expresión. La independencia entre las medidas permite construir crosstabs visualmente ricos sin duplicar la estructura de la matriz.

TEXT

CROSSTAB CON DOS MEDIDAS

	2025		2026	
	Importe	Nº ventas	Importe	Nº ventas
Novela	3500.00	45	2800.00	38
Ensayo	1300.00	18	1100.00	15
Poesía	450.00	8	380.00	7

Cada celda muestra el valor de una medida.
La primera medida es "Importe" con cálculo `Sum`.
La segunda medida es "Nº ventas" con cálculo `Count`.

Qué representa el diagrama: la estructura de un crosstab con dos medidas. Cada celda muestra el valor de una medida para la combinación de fila y columna.

Por qué es relevante: permite construir matrices que muestran varias perspectivas del mismo conjunto de datos sin duplicar la estructura.

Bloque 4 — Las celdas del crosstab

Las celdas del crosstab se declaran con el elemento `crosstabCell`. Cada celda contiene los elementos que se muestran en una posición específica de la matriz. Los elementos más habituales son `textField` con la expresión `$V{NombreMedida}`. El atributo `height` del `crosstabCell` define la altura de la celda. El atributo `width` define el ancho. La celda puede contener varios elementos si se necesita mostrar más de un valor. El motor reproduce la celda en cada posición de la matriz según los grupos de fila y columna.

```
<crosstabCell height="20" width="80">
  <textField pattern="#.##0.00 €">
    <reportElement x="0" y="0" width="80" height="20" uuid="..." />
    <textElement textAlignment="Right" verticalAlignment="Middle">
      <font fontName="DejaVu Sans" size="9"/>
    </textElement>
    <textFieldExpression><![CDATA[$V{ImporteCross}]]></textFieldExpression>
  </textField>
</crosstabCell>
```

XML

Línea 1	<code><crosstabCell height="20" width="80"></code> → declara la celda con 20 píxeles de altura y 80 píxeles de ancho.
Línea 2	<code><textField pattern="#.##0.00 €"></code> → declara el campo de texto con el patrón numérico.
Línea 3	<code><reportElement x="0" y="0" width="80" height="20" uuid="..." /></code> → posición y tamaño del elemento dentro de la celda.
Línea 4-6	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → alineación del texto.
Línea 7	<code><textFieldExpression><![CDATA[\$V{ImporteCross}]]></textFieldExpression></code> → expresión que referencia la medida.
Línea 8	<code></textField></code> → cierra el campo.

El crosstab admite varios tipos de celdas. La celda `crosstabCell` define la celda genérica que se aplica a todas las intersecciones. El crosstab también admite celdas específicas para los encabezados de fila y de columna mediante los elementos `rowGroup` y `columnGroup` que contienen elementos `crosstabRowHeader` y `crosstabColumnHeader`. Estas celdas contienen el texto que aparece en los encabezados de fila y de columna. La personalización de las celdas de encabezado permite construir matrices con encabezados visualmente ricos.

TIPOS DE CELDAS EN UN CROSTAB

```
crosstabRowHeader      → Encabezado de fila (etiqueta del grupo)
crosstabColumnHeader   → Encabezado de columna (etiqueta del grupo)
crosstabCell           → Celda de medida (valor de la intersección)
crosstabTotalRowHeader → Encabezado de la fila de totales
crosstabTotalColumnHeader → Encabezado de la columna de totales
```

TEXT

Qué representa el diagrama: los tipos de celdas disponibles en un crosstab. Cada tipo corresponde a una posición de la matriz.

Por qué es relevante: permite personalizar cada parte del crosstab según el efecto visual deseado.

Bloque 5 — Estilos y compilación del crosstab

Los estilos de un crosstab son estilos JasperReports normales. Se declaran con `<style>` en el informe (o se importan desde una plantilla) y se aplican a `<cellContents style="...">`. No existe en JasperReports 6.20.0 el bloque `crosstabStyle` descrito por la fuente original.

```
<style name="M5CrossHeader" style="Dato" mode="Opaque"
  backgroundColor="#EAF2F8" forecolor="#173F6B" isBold="true"/>
...
<crosstabRowHeader>
  <cellContents style="M5CrossHeader">...</cellContents>
</crosstabRowHeader>
```

XML

El crosstab también forma parte de `informe_ventas.jasper`; no genera un `_crosstab_1.jasper` separado. El E2E verifica tanto la presencia del `.jasper` principal como la ausencia de ese falso artefacto auxiliar.

Contrato ejecutable de las dos medidas del crosstab

El checkpoint 5.5 usa dos medidas reales y complementarias. `ImporteCross` suma el field `importe_cross`, mientras `VentasCross` suma el contador unitario `ventas_cross`. Ambas se calculan para la misma combinación `CategoriaCross × AñoCross`.

XML

```
<measure name="ImporteCross" class="java.lang.Double" calculation="Sum">
  <measureExpression><![CDATA[{F{importe_cross}}]></measureExpression>
</measure>
<measure name="VentasCross" class="java.lang.Integer" calculation="Sum">
  <measureExpression><![CDATA[{F{ventas_cross}}]></measureExpression>
</measure>
```

La celda de detalle muestra las dos variables generadas por JasperReports: `$V{ImporteCross}` con formato monetario y `$V{VentasCross}` + " ventas". El mismo patrón se repite en el total de fila, el total de columna y el total general.

TEXT

```
detalle                -> ImporteCross + VentasCross
rowTotalGroup          -> total por CategoriaCross
columnTotalGroup       -> total por AnioCross
rowTotalGroup + columnTotal -> total general
```

Esto significa que `VentasCross` no es una segunda tabla ni una segunda `crosstabCell` independiente: es una segunda medida dentro de las mismas celdas. La fuente `DatasetCrosstabVentas` aporta `ventas_cross = 1` por venta y la medida `Sum` convierte esos unos en el número de operaciones de cada cruce.

El contrato completo del checkpoint queda, por tanto, formado por `DatasetCrosstabVentas`, los grupos `CategoriaCross` y `AnioCross`, y las medidas `ImporteCross` y `VentasCross`. Esos mismos nombres aparecen en la Parte A, en el JRXML byte a byte de Parte B y en la ejecución E2E.

Resumen rápido de la teoría

- El crosstab es una matriz que organiza los datos en filas, columnas y medidas.
- Se declara directamente con el elemento nativo `crosstab`.
- Los grupos de fila y de columna se declaran con `rowGroup` y `columnGroup`.
- La expresión `bucketExpression` determina los valores distintos de cada grupo.
- Las medidas se declaran con `measure` y contienen su cálculo y su expresión.
- Las celdas se declaran con `crosstabCell` y contienen los campos de la medida.
- Los estilos son estilos JasperReports normales aplicados a `cellContents`.
- El crosstab se compila dentro de `informe_ventas.jasper`; no genera un `.jasper` independiente.

- El crosstab ejecutable calcula simultáneamente `ImporteCross` y `VentasCross` para detalle y totales.

Punto 5.6 — Estilos y plantillas

Objetivos de aprendizaje

- Comprender el concepto de plantilla de estilo externa y su formato .jrtx.
- Crear una plantilla de estilo con Jaspersoft Studio y con edición manual del XML.
- Importar una plantilla de estilo en un informe mediante el elemento `template`.
- Aplicar los estilos de la plantilla a elementos, bandas y componentes.
- Combinar estilos de plantilla con estilos locales declarados en el informe.
- Documentar las plantillas de estilo del proyecto EditorialReports.

Parte teórica

Bloque 1 — El concepto de plantilla de estilo externa

Una plantilla de estilo externa es un archivo con extensión .jrtx que contiene un conjunto de estilos reutilizables en varios informes. A diferencia de los estilos declarados dentro de un JRXML, que solo están disponibles en ese informe, los estilos de una plantilla pueden importarse desde cualquier informe del proyecto. Esta característica permite centralizar la definición de estilos y aplicar la misma presentación a varios informes sin duplicar la declaración. La plantilla es un archivo XML con la misma estructura que el bloque de estilos del JRXML, pero sin el elemento raíz `jasperReport`. Su contenido se compone únicamente de elementos `style` con sus propiedades.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template">
  <style name="TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#1A3D6B"/>
  <style name="TituloSecundario" fontName="DejaVu Sans" fontSize="14" isBold="true" forecolor="#4A6B8A"/>
</jasperTemplate>
```

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → declaración XML obligatoria del archivo de plantilla.
Línea 2	<code><jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template"></code> → elemento raíz de la plantilla. La plantilla usa el namespace específico <code>http://jasperreports.sourceforge.net/jasperreports/template</code> , distinto del namespace raíz de un JRXML de informe.
Línea 3	<code><style name="TituloPrincipal" ...></code> → declara un estilo con su nombre y sus propiedades. El estilo estará disponible en cualquier informe que importe la plantilla.
Línea 4	<code><style name="TituloSecundario" ...></code> → declara el segundo estilo.
Línea 5	<code></jasperTemplate></code> → cierra el elemento raíz.

La plantilla de estilo se almacena como un archivo independiente en la carpeta de recursos del proyecto. La convención habitual es situarla en una subcarpeta denominada `styles` o en la raíz de la carpeta de recursos. El archivo se versiona junto con los informes y se comparte entre todos los informes del proyecto. Cuando un informe importa la plantilla, el motor carga los estilos y los pone disponibles para los elementos del informe. Los estilos de la plantilla pueden ser referenciados por nombre igual que los estilos locales. La diferencia es que su definición reside en el archivo externo y puede modificarse sin tocar los informes que la utilizan.

TEXT

ESTRUCTURA DE PLANTILLA Y USO

```
EditorialReports/
├── resources/
│   └── styles/
│       └── EditorialStyles.jrtx  ← plantilla externa
informe_ventas.jrxml:
<template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template>
...
<staticText>
  <reportElement ... style="TituloPrincipal"/>
  ...
</staticText>
```

El informe importa la plantilla y usa los estilos como si fueran locales.
Un cambio en la plantilla se refleja en todos los informes que la usan.

Qué representa el diagrama: la ubicación de la plantilla y su uso desde un informe. El informe importa la plantilla y referencia los estilos por nombre.

Por qué es relevante: permite centralizar la definición de estilos y aplicar la misma presentación a varios informes sin duplicar la declaración.

Bloque 2 — Creación de la plantilla de estilo

Una plantilla de estilo se crea como un archivo .jrtx en la carpeta de recursos del proyecto. Jaspersoft Studio incluye un asistente para crear plantillas desde el menú File > New > Jasper Template. El asistente genera un archivo vacío con el elemento raíz `jasperTemplate` y permite añadir estilos desde la interfaz. La plantilla también puede crearse manualmente escribiendo el XML con un editor de texto. Ambas formas producen el mismo resultado: un archivo XML con la declaración de los estilos y sus propiedades. La plantilla se guarda en la carpeta `styles` del proyecto y se referencia desde los informes.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperTemplate xmlns="http://jasperreports.sourceforge.net/jasperreports/template">
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10" bold="false" italic="false" underline="false"
  strikeThrough="false"/>
  <style name="TituloPrincipal" parent="Sans_Normal" fontSize="18" isBold="true" forecolor="#1A3D6B"/>
  <style name="TituloSecundario" parent="Sans_Normal" fontSize="14" isBold="true" forecolor="#4A6B8A"/>
  <style name="TextoTablaCabecera" parent="Sans_Normal" fontSize="10" isBold="true" forecolor="#FFFFFF" backcolor="#4A6B8A"
  mode="Opaque"/>
  <style name="TextoTabla" parent="Sans_Normal" fontSize="10"/>
  <style name="TextoPequeño" parent="Sans_Normal" fontSize="9" isItalic="true" forecolor="#666666"/>
</jasperTemplate>
```

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → declaración XML.
Línea 2	<code><jasperTemplate xmlns="..."></code> → elemento raíz.
Línea 3	<code><style name="Sans_Normal" isDefault="true" .../></code> → estilo por defecto de la plantilla. Todos los estilos hijos heredan de él.
Línea 4	<code><style name="TituloPrincipal" parent="Sans_Normal" .../></code> → estilo para títulos principales.
Línea 5	<code><style name="TituloSecundario" parent="Sans_Normal" .../></code> → estilo para títulos secundarios.
Línea 6	<code><style name="TextoTablaCabecera" parent="Sans_Normal" .../></code> → estilo para las cabeceras de tabla.
Línea 7	<code><style name="TextoTabla" parent="Sans_Normal" .../></code> → estilo para las celdas de tabla.
Línea 8	<code><style name="TextoPequeño" parent="Sans_Normal" .../></code> → estilo para textos pequeños.
Línea 9	<code></jasperTemplate></code> → cierra el elemento raíz.

La plantilla puede contener estilos con estilos condicionales y estilos con herencia. La organización en jerarquía es la misma que en el JRXML. La única diferencia es que el elemento raíz es `jasperTemplate` en lugar de `jasperReport`. Los estilos declarados en la plantilla no pueden referenciar variables, parámetros ni campos del informe: solo pueden referenciar otros estilos de la misma plantilla. Esta restricción garantiza que la plantilla sea autocontenida y pueda importarse desde cualquier informe. Las condiciones de los estilos condicionales que necesiten campos o parámetros deben declararse en el informe, no en la plantilla.

TEXT

RESTRICCIONES DE LA PLANTILLA

Permitido en la plantilla:

- Estilos con propiedades tipográficas.
- Estilos con herencia mediante `parent`.
- Estilos condicionales con expresiones literales.
- Estilos con bordes y colores.

Prohibido en la plantilla:

- Referencias a `$F{}` (campos del informe).
- Referencias a `$P{}` (parámetros del informe).
- Referencias a `$V{}` (variables del informe).
- Referencias a `subdatasets`.

La plantilla es autocontenida y agnóstica del informe.

Qué representa el diagrama: las restricciones de la plantilla. Los estilos no pueden referenciar campos, parámetros ni variables del informe.

Por qué es relevante: permite diseñar plantillas que puedan importarse desde cualquier informe sin errores de resolución.

Bloque 3 — Importación de la plantilla en el informe

La importación de una plantilla en un informe se realiza con el elemento `template` que se declara dentro del elemento raíz `jasperReport` y antes de cualquier otro contenido. El elemento contiene una expresión que devuelve la ruta del archivo `.jrtx`. La ruta puede ser relativa al directorio de ejecución del programa o absoluta. La expresión se encierra en un bloque `CDATA`. Un informe puede importar varias plantillas. Los estilos de las plantillas importadas están disponibles en todo el informe y pueden referenciarse por nombre desde cualquier elemento. Si dos plantillas declaran un estilo con el mismo nombre, el último importado prevalece.

XML

```
<jasperReport xmlns="..." name="informe_ventas">
  <template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template>
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="Sans_Normal" isDefault="true" .../>
  ...
</jasperReport>
```

Línea 1	<code><jasperReport xmlns="..." name="informe_ventas"></code> → elemento raíz del informe.
Línea 2	<code><template><![CDATA["resources/styles/EditorialStyles.jrtx"]]></template></code> → importa la plantilla de estilo. La expresión devuelve la ruta del archivo <code>.jrtx</code> .
Línea 3	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → propiedad del adaptador de datos.
Línea 4	<code><style name="Sans_Normal" isDefault="true" .../></code> → estilo local del informe. Puede sobrescribir el estilo de la plantilla o definir uno nuevo.

El elemento `template` debe declararse antes de las declaraciones de estilos locales y antes de las bandas. El orden de importación determina la precedencia: los estilos de las plantillas importadas se aplican primero y los estilos locales pueden sobrescribirlos. Si un informe declara un estilo con el mismo nombre que un estilo de la plantilla, el estilo local prevalece. Esta característica permite que un informe ajuste la presentación sin modificar la plantilla común. La combinación de plantillas y estilos locales es la que permite construir informes visualmente coherentes y con ajustes específicos.

TEXT

PRECEDENCIA DE ESTILOS

- Estilos declarados en la plantilla.
Se aplican primero.
 - Estilos declarados en el informe.
Sobrescriben los estilos de la plantilla con el mismo nombre.
 - Propiedades declaradas directamente en el elemento.
Sobrescriben los estilos de la plantilla y del informe.
- Orden de aplicación: plantilla → informe → elemento.

Qué representa el diagrama: la precedencia de estilos desde la plantilla hasta el elemento. El elemento tiene la máxima prioridad.

Por qué es relevante: permite decidir dónde colocar cada propiedad según el nivel de personalización que se necesite.

Bloque 4 — Aplicación de estilos de plantilla a componentes

Los estilos de la plantilla se aplican a elementos y a las celdas internas de componentes. Los elementos `staticText`, `textField`, `image`, `line`, `rectangle` y `frame` admiten el atributo `style` con el nombre del estilo. Los componentes reutilizan estilos JasperReports en sus elementos internos. En una tabla se aplica el estilo a `c:columnHeader` o `c:detailCell`; en un `crosstab`, a `cellContents`. No existe un bloque genérico `tableStyle` o `crosstabStyle` en JasperReports 6.20.0. La coherencia de nombres entre la plantilla y el componente es condición necesaria para que el estilo se aplique.

XML

```
<staticText>
  <reportElement x="0" y="0" width="555" height="30" style="TituloPrincipal"/>
  <text><![CDATA[Informe de Ventas]]></text>
</staticText>

<textField>
  <reportElement x="0" y="0" width="100" height="20" style="TextoTabla"/>
  <textFieldExpression><![CDATA[$F{importe_total}]]></textFieldExpression>
</textField>
```

Línea 1-4	<code>staticText</code> con el estilo <code>TituloPrincipal</code> referenciado desde el atributo <code>style</code> del bloque <code>reportElement</code> . El estilo se aplica al texto del título.
------------------	---

Línea 6-9

textField con el estilo TextoTabla referenciado desde el atributo style. El estilo se aplica al campo del importe.

Los estilos de la plantilla pueden sobrescribirse elemento por elemento. Si un elemento necesita un color distinto al de la plantilla, se declara la propiedad directamente en el elemento y esta prevalece. La combinación de estilos de plantilla con propiedades específicas permite construir informes visualmente coherentes y con ajustes puntuales. La buena práctica consiste en mantener la mayor parte de las propiedades en la plantilla y limitar las sobrescrituras a los casos en que la presentación específica lo requiera. Esta aproximación reduce la duplicación y facilita los cambios globales.

TEXT

APLICACIÓN DE ESTILOS DE PLANTILLA

Elemento con estilo de plantilla:

```
<staticText>
  <reportElement ... style="TituloPrincipal"/>
  <text>...</text>
</staticText>
→ Hereda todas las propiedades del estilo.
```

Elemento con estilo y sobrescritura:

```
<staticText>
  <reportElement ... style="TituloPrincipal"/>
  <textElement forecolor="#FF0000"/>
  <text>...</text>
</staticText>
→ Hereda el estilo pero sobrescribe el color.
```

Componente con estilo de plantilla:

```
<c:columnHeader style="TextoTablaCabecera" height="20">...</c:columnHeader>
→ Aplica el estilo de la plantilla al encabezado de la tabla.
```

Qué representa el diagrama: los tres casos de aplicación de estilos de plantilla. El elemento puede heredar solo el estilo, heredar y sobrescribir, o aplicar el estilo a un componente.

Por qué es relevante: permite decidir el nivel de personalización de cada elemento sin duplicar la definición del estilo.

Bloque 5 — Buenas prácticas en el uso de plantillas

El uso de plantillas de estilo en un proyecto profesional sigue cuatro buenas prácticas. La primera es centralizar en la plantilla todos los estilos que comparten varios informes. La segunda es limitar la plantilla a las propiedades tipográficas y de borde, dejando las propiedades específicas del informe para los estilos locales. La tercera es documentar la plantilla con un archivo .md que describa los estilos disponibles y su uso. La cuarta es versionar la plantilla junto con los informes para garantizar la coherencia del proyecto. La combinación de las cuatro prácticas construye un sistema de estilos mantenible y coherente.

TEXT

BUENAS PRÁCTICAS EN PLANTILLAS DE ESTILO

1. Centralizar en la plantilla todos los estilos compartidos.
2. Limitar la plantilla a tipografía, color y bordes.
3. Documentar los estilos con un archivo ESTILOS.md.
4. Versionar la plantilla junto con los informes.

Beneficios:

- Cambios globales en un solo archivo.
- Coherencia visual entre informes.
- Reducción de la duplicación.
- Facilidad de mantenimiento a largo plazo.

Qué representa el diagrama: las cuatro buenas prácticas en plantillas de estilo y sus beneficios.

Por qué es relevante: permite construir un sistema de estilos mantenible que facilite la evolución del proyecto.

La plantilla puede evolucionar a lo largo del tiempo. Un cambio en la plantilla afecta a todos los informes que la importan. Este comportamiento es deseable cuando el cambio es una mejora global, pero puede ser problemático si el cambio afecta a la presentación de un informe concreto. La práctica recomendada consiste en versionar la plantilla con un número de versión en el nombre del archivo (EditorialStyles_v1.jrtx, EditorialStyles_v2.jrtx) y en importar la versión específica desde cada informe. Esta aproximación permite evolucionar la plantilla sin afectar a los informes existentes. Los informes nuevos pueden importar la versión más reciente.

TEXT

VERSIONADO DE PLANTILLAS

```
resources/styles/
├── EditorialStyles_v1.jrtx  ← versión inicial
├── EditorialStyles_v2.jrtx  ← versión con nuevos estilos
└── EditorialStyles_v3.jrtx  ← versión actual
```

```
informe_ventas.jrxml:
<template><![CDATA["resources/styles/EditorialStyles_v1.jrtx"]]></template>
```

```
informe_catalogo.jrxml:
<template><![CDATA["resources/styles/EditorialStyles_v3.jrtx"]]></template>
```

Cada informe importa la versión que necesita.

Qué representa el diagrama: el versionado de plantillas. Cada informe importa la versión que necesita sin afectar a los demás.

Por qué es relevante: permite evolucionar la plantilla sin romper la presentación de los informes existentes.

Contrato real de estilos del checkpoint 5.6

La plantilla ejecutable es `resources/styles/EditorialStyles.jrtx` y expone siete estilos que el JRXML referencia por nombre. Este contrato es importante: cambiar un nombre en el JRTX sin cambiar el JRXML provoca que el estilo no pueda resolverse al compilar.

Estilo	Aplicación en <code>informe_catalogo.jrxml</code>
M5TituloPrincipal	título principal del informe
M5GrupoCabecera	cabecera de <code>CategoriaGroup</code>
M5TablaCabecera	<code>c:columnHeader</code> de la tabla
M5TablaDetalle	<code>c:detailCell</code> de la tabla
M5CrosstabCabecera	cabeceras de fila y columna del crosstab
M5CrosstabDetalle	celda de detalle del crosstab
M5CrosstabTotal	cabeceras y celdas de total del crosstab

Los estilos locales heredados siguen coexistiendo con los externos; `Sans_Normal` continúa siendo el único estilo por defecto del informe. La plantilla no introduce un segundo `isDefault="true"`.

Resumen rápido de la teoría

- Una plantilla de estilo es un archivo `.jrtx` con estilos reutilizables.
- Se declara con el elemento raíz `jasperTemplate`.
- Se importa en el informe con el elemento `template`.
- Los estilos de la plantilla están disponibles en todo el informe.
- Las propiedades del elemento sobrescriben las de la plantilla y las del informe.
- La plantilla puede aplicarse a elementos, bandas y componentes.
- La plantilla es autocontenida y no referencia campos, parámetros ni variables.
- Las buenas prácticas incluyen centralizar, documentar y versionar.

- El contrato ejecutable de la plantilla usa `M5TituloPrincipal`, `M5GrupoCabecera`, `M5TablaCabecera`, `M5TablaDetalle`, `M5CrosstabCabecera`, `M5CrosstabDetalle` y `M5CrosstabTotal`.